1. What is DCL in SQL?

DCL (Data Control Language) is the part of SQL used to **control access to data** in the database.
It deals with:

- Who can **see** data
- Who can **change** data
- Who can **grant permissions to others**

Core DCL commands:

- GRANT – give permissions
- REVOKE – remove permissions
- (DENY – special to SQL Server)

2. Basic Concepts You Need Before DCL

DCL works with these things:

- **Users** – actual logins (e.g., user1, john, app_user)
- **Roles** – group of privileges that can be given to users
- **Privileges / Permissions**
    - On **objects**: SELECT, INSERT, UPDATE, DELETE, EXECUTE, etc.
    - Sometimes **system-level**: CREATE USER, BACKUP, etc.
- **Objects** – tables, views, sequences, procedures, functions, schemas.

3. GRANT – Giving Permissions

3.1 Generic Syntax

GRANT privilege_list

ON object_name

TO grantee

[WITH GRANT OPTION];

- privilege_list → SELECT, INSERT, UPDATE, etc.
- object_name → schema.table, view, procedure, etc.
- grantee → user or role

- WITH GRANT OPTION → allows them to further grant same permissions to others

---

3.2 MySQL

1) Create a user (admin only)

CREATE USER 'report_user'@'%' IDENTIFIED BY 'StrongPassword123!';

2) Grant SELECT on one table

GRANT SELECT

ON sales_db.orders

TO 'report_user'@'%';

Now report_user can run:

SELECT * FROM sales_db.orders;

…but **cannot** insert/update/delete.

3) Grant multiple privileges on a table

GRANT SELECT, INSERT, UPDATE

ON sales_db.customers

TO 'report_user'@'%';

4) Grant ALL privileges on a database

GRANT ALL PRIVILEGES

ON sales_db.*

TO 'admin_user'@'%';

5) Grant with GRANT OPTION

GRANT SELECT

ON sales_db.orders

TO 'manager'@'%'

WITH GRANT OPTION;

Now manager can do:

GRANT SELECT ON sales_db.orders TO 'analyst'@'%';

In MySQL, privileges are usually finalized with

4. MySQL

Revoke a specific privilege

REVOKE SELECT

ON sales_db.orders

FROM 'report_user'@'%';

Now report_user cannot run SELECT on that table.

Revoke all privileges on a database

REVOKE ALL PRIVILEGES

ON sales_db.*

FROM 'report_user'@'%';

Again, typically

5.Permissions on a **specific object**:

1. SELECT (read rows)
2. INSERT (add rows)
3. UPDATE (modify rows)
4. DELETE (remove rows)
5. EXECUTE (run a procedure/function)
6. REFERENCES (foreign key references)
7. etc.

Example :

GRANT CREATE, DROP, RELOAD, SHUTDOWN, PROCESS

ON *.*

TO 'admin_user'@'%'

WITH GRANT OPTION;

5.1 MySQL Example

CREATE ROLE 'reporting_role';

GRANT SELECT

ON sales_db.orders

TO 'reporting_role';

GRANT 'reporting_role' TO 'report_user'@'%';

SET DEFAULT ROLE 'reporting_role' TO 'report_user'@'%';

---

6. Real-World Scenarios

Scenario 1: Application User With Limited Rights (MySQL)

App user can only **SELECT, INSERT, UPDATE** on specific tables.

CREATE USER 'app_user'@'%' IDENTIFIED BY 'AppPass123!';

GRANT SELECT, INSERT, UPDATE

ON sales_db.customers

TO 'app_user'@'%';

GRANT SELECT, INSERT

ON sales_db.orders

TO 'app_user'@'%';

Scenario 2: Manager Can Grant SELECT to Others (Grant Option)

GRANT SELECT

ON sales_db.orders

TO 'manager'@'%'

WITH GRANT OPTION;

Now:

-- run by manager

GRANT SELECT ON sales_db.orders TO 'analyst'@'%';

If you want to stop manager from further granting:

```
REVOKE GRANT OPTION FOR SELECT
ON sales_db.orders
FROM 'manager'@'%';
```

DATETIME Functions in SQL (Basic → Advanced)

7. Getting Current Date & Time

MySQL

```
SELECT CURDATE();     -- Returns current date (YYYY-MM-DD)
SELECT CURTIME();     -- Returns current time (HH:MM:SS)
SELECT NOW();         -- Returns current date & time
```

SQL Server

```
SELECT GETDATE();     -- Current date & time
SELECT CURRENT_TIMESTAMP;
SELECT GETUTCDATE();   -- UTC time
```

Extracting Parts of Date/Time

MySQL

```
SELECT
  YEAR(NOW()) AS year,
  MONTH(NOW()) AS month,
  DAY(NOW()) AS day,
  HOUR(NOW()) AS hour,
  MINUTE(NOW()) AS minute,
  SECOND(NOW()) AS second;
```

SQL Server

```
SELECT
  YEAR(GETDATE()),
```

MONTH(GETDATE()),

DAY(GETDATE());

---

8. Formatting Date & Time

MySQL

```
SELECT DATE_FORMAT(NOW(), '%d-%m-%Y %h:%i:%s');
```

9. Adding / Subtracting Date & Time

MySQL

```
SELECT DATE_ADD(NOW(), INTERVAL 7 DAY);    -- Add 7 days
SELECT DATE_SUB(NOW(), INTERVAL 2 MONTH);   -- Subtract 2 months
```

5. Difference Between Dates

MySQL

```
SELECT DATEDIFF('2025-12-31', '2025-12-01');   -- Days difference
```

10. Convert String to Date

MySQL

```
SELECT STR_TO_DATE('31-12-2025', '%d-%m-%Y');
```

11. Useful Date Functions Summary (MySQL)

Function	Description
NOW()	Current date & time
CURDATE()	Today date
CURTIME()	Current time
DATE()	Extract date from datetime
TIME()	Extract time
DATEDIFF()	Difference in days
DATE_ADD()	Add time units
DATE_SUB()	Subtract time units

Function	Description
DATE_FORMAT()	Format date
STR_TO_DATE()	Convert string to date

---

12. Practical Examples

Example 1: Get all users registered in last 30 days

```
SELECT * FROM users
WHERE registration_date >= DATE_SUB(NOW(), INTERVAL 30 DAY);
```

Example 2: Find age of a person

```
SELECT TIMESTAMPDIFF(YEAR, '2000-05-10', CURDATE()) AS age;
```

Example 3: Group sales by month

```
SELECT
  DATE_FORMAT(order_date, '%Y-%m') AS month,
  SUM(amount) AS total_sales
FROM orders
GROUP BY DATE_FORMAT(order_date, '%Y-%m');
```

Example 4: Convert datetime to only date

```
SELECT DATE(order_datetime) FROM orders;
```

Giving permission on selected (specific) columns to another user is possible in some SQL databases, but NOT all support column-level privileges.

MySQL

MySQL allows granting SELECT, INSERT, UPDATE on specific columns.

✓ Syntax

GRANT privilege (column_list)

ON table_name

TO 'username'@'host';

Example: Give SELECT permission only for specific columns

Suppose your table is:

employees(id, name, salary, email)

You want user report_user to see only name and email (not salary).

GRANT SELECT (name, email)

ON company.employees

TO 'report_user'@'%';

Now the user can run:

SELECT name, email FROM company.employees;

But this will fail:

SELECT salary FROM company.employees; -- Access denied
